Branch Coverage Report

Project: File Compression Tool (Basic ZIP Implementation): Team 14
Date: 05-Nov-2026
Code version: `main` @ `cae72ce` + Sprint-3 tests
Tool: GCC 13.3.0 `--coverage` (gcov) + gcovr 8.8, `--txt-metric branch`
Detailed per-line report: `Documents/Test_Reports/coverage/index.html` (branch markers on each line)

---

1. Result

Metric	Taken	Total	Coverage
Branches	**401**	**487**	**82.3%**

Per file

File	Module	Branches taken	Branch coverage
`src/frequency.cpp`	Compression Engine (REQ-2)	2 / 2	**100%**
`src/huffman.cpp`	Huffman Tree Module (REQ-3)	57 / 61	**93.4%**
`src/codec.cpp`	Compression / Decompression Engine (REQ-4 to REQ-7)	222 / 269	**82.5%**
`src/main.cpp`	CLI (REQ-1)	87 / 112	**77.7%**
`src/file_input.cpp`	File I/O (REQ-1)	33 / 43	**76.7%**
Total		**401 / 487**	**82.3%**

A branch counts as covered when **both** outcomes of a decision (true and false) have been executed at least once by the test suite.

---

2. Method and settings

The same instrumented run as the Code Coverage Report: 36 unit tests, 21 system validation test cases and 17 lossless/corruption checks, measured on `src/` only.

gcovr option	Why it is used
`--exclude-throw-branches`	The C++ compiler adds a hidden "exception thrown" branch to almost every call that could throw (`std::string` , file streams, `new`). These are not decisions in our code. Counting them would make most "branches" untestable library internals

gcovr option	Why it is used
`--exclude-unreachable-branches`	Removes branches the compiler generates for lines that contain no real code

Reproduce: `bash tests/coverage.sh` (output in `Documents/Test_Reports/coverage/branches.txt`).

---

3. Improvement in Sprint 3

	Branches	Coverage
Before Sprint-3 tests	346 / 487	71.0%
After `tests/test_robustness.cpp` and TC-DEC-05	401 / 487	**82.3%**

Branches newly covered (both outcomes now tested):

- truncated header at each of the three header fields (UT-ROB-02/03/04)
- frequency overflow (UT-ROB-05)
- empty and non-empty bit-count consistency (UT-ROB-06/07)
- single-symbol bit count and payload checks (UT-ROB-08/09)
- incomplete code and too many symbols (UT-ROB-10/11)
- truncated payload on disk (UT-ROB-12)
- unreadable input and missing output folders (UT-ROB-14/15/16)
- overflow and invalid-code guards in `huffman.cpp` (UT-ROB-17/18)
- default output name for archives without `.huff` (TC-DEC-05)

---

4. Branches not covered (86) and why

File : line(s)	Decision	Outcome not tested	Reason
`codec.cpp` 73–75, 310–312	`if (!out.good())` after writing	true (write failed)	Needs a full disk or a device failure
`codec.cpp` 82–84, 318–320	`if (ec)` after renaming the temp file	true (rename failed)	Needs the OS to refuse a rename in a writable folder
`codec.cpp` 325–326	`if (ec || written_size != original_size)`	true	Only on a disk error after a successful write
`codec.cpp` 159–160, 163–164	Tree rebuild throws / tree missing	true	**Unreachable.** The earlier frequency checks guarantee a valid tree (see the Code Coverage Report, Section 4)
`codec.cpp` 178	`root->frequency != original_size`	true	**Unreachable.** With one symbol, its frequency always equals the validated original size. The other half of the condition (bit count) is tested by UT-ROB-08

File : line(s)	Decision	Outcome not tested	Reason
`codec.cpp` 217, 270–273	`if (!node)` while building the lookup table / decoding	true	**Unreachable.** A Huffman tree is a full binary tree, so a child is never missing
`codec.cpp` 255–258	"Exceeds original size" in the fast lookup path	true	Needs crafted extra symbols that start on a byte boundary. The same check in the bit-by-bit path is covered (UT-ROB-11)
`codec.cpp` 292	`current != root || bytes_written != original_size`	second condition true alone	When decoding ends on a symbol boundary with the wrong count, an earlier check has already rejected it
`file_input.cpp` 24–25	`if (!in)` in `validate_input_file`	true	Permission-denied file (not testable as an administrator or on Windows)
`file_input.cpp` 41–42, 46–47	`size < 0`; read fails	true	Stream or device error while reading
`huffman.cpp` 146, 149	Bit count reaches 2^{64}	true	Needs more than 2 exabytes of output
`huffman.cpp` 104	`prefix.empty()` for a leaf	(partial)	The empty-prefix case only happens for a single-symbol tree. It is covered by UT-HUF-02, but gcov counts an extra string-construction branch
`main.cpp` 30	`fs::exists(b) && fs::equivalent(a, b)`	equivalent paths written differently (`./a.txt` vs `a.txt`)	Behaviour checked manually during Sprint 2 (D-14). The automated test uses identical paths
`main.cpp` 34, 39, 77	Short-string checks inside `ends_with` / argument parsing	some combinations	Branches created by short-circuit `&&` with standard-library calls; every user-visible case (TC-ERR-06, TC-DEC-04/05) is tested
`main.cpp` 93–96	`catch (bad_alloc)` / `catch (exception)`	entered	Needs the program to run out of memory

Conclusion: every decision that a user or a damaged archive can trigger has both outcomes tested. The remaining uncovered branches are hardware or OS failure paths, out-of-memory safety nets, and defensive checks that are unreachable because earlier validation already handles the case.

Appendix: gcovr text output (branches)

```
------------------------------------------------------------------------------
                           GCC Code Coverage Report
Directory: .
------------------------------------------------------------------------------
File                                       Branches    Taken  Cover   Missing
------------------------------------------------------------------------------
src/codec.cpp                                 269      222    82%
73,74,75,82,83,84,159,160,163,164,178,217,255,256,257,258,270,271,272,273,292,310,3
src/file_input.cpp                             43       33    76%
24,25,41,42,46,47
src/frequency.cpp                               2        2   100%
src/huffman.cpp                                61       57    93%   104,146,149
src/main.cpp                                  112       87    77%
30,34,39,77,93,94,96
------------------------------------------------------------------------------
TOTAL                                         487      401    82%
------------------------------------------------------------------------------
```